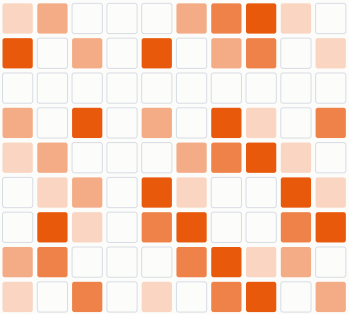
LECTURE NOTES · EFFICIENCY, 2026

Sparsity and Precision, Visually

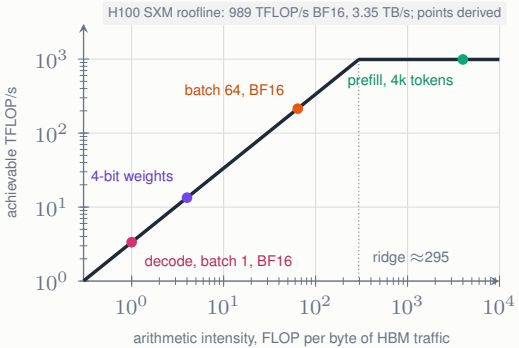
The two levers that cut the cost of a language model: mixture of experts, pruning, sparse activations and sparse attention; number formats, low-precision training, precision scaling laws, and quantization for inference.



blank: pruned four shades: 2-bit values

Two levers on one cost model

Sparsity cuts how many numbers each token touches; precision cuts how many bits each number has.



Decoding one token reads every active weight once for 2 FLOPs: at 16 bits that is 1 FLOP per byte, 300× below the ridge. Fewer or smaller weights move the point; batching moves it right.

The accounting

training FLOPs $\approx 6\, N_{\text{active}}\, D$; decode FLOPs per token $\approx 2\, N_{\text{active}}$
weight bytes $= N_{\text{total}} \cdot \text{bits}/8$; KV bytes per token $= 2\, L\, h_{kv}\, d_h \cdot \text{bits}/8$

Sparsity lowers N_{active} (MoE, pruning, silent neurons) or the KV entries read (sparse attention) while leaving N_{total}, the model's capacity, large.

Precision lowers the bits: each halving doubles tensor-core throughput on hardware that supports it (BF16 → FP8 → FP4) and halves the bytes moved, which is what decode is waiting on.

The catch is the same for both: quality lost to rounding or removal, and the need for hardware or kernels that skip what was removed.

The map, and a timeline

Four kinds of sparsity, four questions about precision, and when each arrived in language models.

Sparsity: which numbers are touched

Mixture of experts
trained in: a router picks k of E experts per token

Weight pruning
after training: zero out weights, structured or not

Activation sparsity
per input: most MLP neurons are silent; skip them

Attention sparsity
per query: read a subset of the KV cache

Precision: how many bits per number

Formats
exponent vs mantissa; integers; block scales (MX, NVFP4)

Training
mixed precision, FP8 with tile scales, FP4 with Hadamard and SR

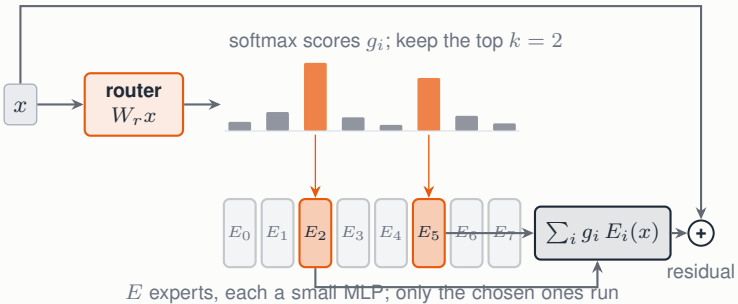
Laws
effective parameters per bit; the 7–8 bit optimum; 4-bit at inference

Quantization
weight-only, W8A8, rotations, QAT, KV cache, MoE weights



The mixture-of-experts layer

Replace one wide MLP by E narrow ones and a router; each token uses k of them, so FLOPs follow k, not E.



$$y = x + \sum_{i \in \mathrm{TopK}(g)} g_i\, E_i(x), \qquad g = \mathrm{softmax}(W_r x)$$

Counting

N_{total}: all experts; what fits in memory and what the model knows.

N_{active}: the k experts plus attention and embeddings; what one token computes.

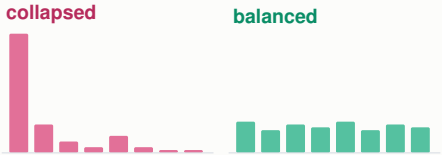
DeepSeek-V3: 671B total, 37B active, 1 in 18. Dense Llama 3 405B: 405B both.

Quality tracks N_{total} at a discount; cost tracks N_{active}: that gap is the whole point.

DeepSeek-V3 and Kimi K2 use a sigmoid gate, normalized over the chosen k; the router is one small matrix per layer.

Routing: keeping the experts busy, and fed

Left alone, a router sends most tokens to a few experts. Every recipe adds a pressure toward balance.



tokens per expert in one batch; collapse wastes $E - 1$ experts and overflows the one in use

Auxiliary loss (Switch): $\mathcal{L}_{\text{aux}} = \alpha E \sum_i f_i P_i$, the fraction of tokens f_i times the mean router probability P_i; minimal when uniform; $\alpha \approx 0.01$. A **router z-loss** (ST-MoE) keeps the logits small.

Aux-loss-free (DeepSeek-V3): add a bias b_i to expert i's score for the top-k choice only; after each step, $b_i \mathrel{-}= \gamma$ if overloaded, $b_i \mathrel{+}= \gamma$ if underloaded. Nothing competes with the language loss.

Capacity, dropping, and who picks whom

Static kernels want a fixed number of tokens per expert: capacity $C = \text{CF} \cdot T/E$ with capacity factor CF ≈ 1–1.25. Tokens past capacity are *dropped*: they skip the layer and continue on the residual. Dropless MoE (MegaBlocks) uses block-sparse matmuls, so experts take variable-size batches and nothing is dropped.

Expert choice (Zhou et al. 2022): each expert picks its top tokens; exact balance by construction, but a token's experts then depend on the batch, awkward at decode.

Top-k choices. Switch: $k = 1$; GShard, Mixtral: $k = 2$; DeepSeek-V3, Qwen3, Kimi K2: $k = 8$ of 128–384 fine-grained experts; gpt-oss: 4 of 128. Larger k with smaller experts gives more combinations per FLOP.

Fine-grained experts, shared experts, and who uses what

Same FLOPs, far more combinations; and the active fraction fell from a quarter to a thirtieth in two years.

coarse: 8 experts, top-2



$\binom{8}{2} = 28$ combinations; G = expert width / dense width

fine-grained: 32 experts of 1/4 the width, top-8



shared, always on

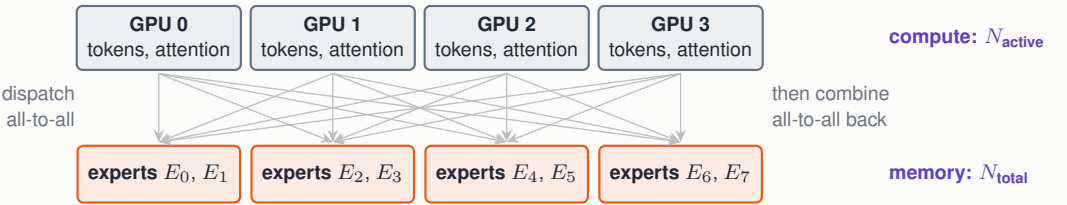
$\binom{32}{8} = 10{,}518{,}300$ combinations at the same active FLOPs (DeepSeekMoE, 2024)

Model (date)	total	active	active/total	experts	per token	notes
Mixtral 8×7B (Dec 2023)	47B	13B	28%	8	2	coarse experts, sliding-window attention
DeepSeek-V3 (Dec 2024)	671B	37B	5.5%	256+1 shared	8+1	MLA, aux-loss-free balance, FP8 training
Llama 4 Maverick (Apr 2025)	400B	17B	4.3%	128+1 shared	1+1	alternating dense and MoE layers
Qwen3-235B-A22B (Apr 2025)	235B	22B	9.4%	128	8	no shared expert
Kimi K2 (Jul 2025)	1.04T	32B	3.1%	384+1 shared	8+1	MLA; MuonClip
gpt-oss-120b (Aug 2025)	117B	5.1B	4.4%	128	4	MXFP4 expert weights; one 80 GB GPU
Qwen3-Next-80B-A3B (Sep 2025)	80B	3B	3.8%	512+1 shared	10+1	hybrid linear attention

active/total is derived from the stated counts; expert counts are per MoE layer. 2026 continues the trend: DeepSeek-V4-Pro 1.6T/49B, Kimi K3 2.8T/104B with 16 of 896 experts plus 2 shared†.

MoE systems: expert parallelism, all-to-all, and the memory bill

The experts live on different GPUs; every MoE layer is two network shuffles around a small matmul.



each token travels to its k experts' GPUs and back, twice per layer; DeepSeek-V3 caps each token at $M = 4$ nodes and overlaps the shuffles with compute (DualPipe).

Memory, not FLOPs

All experts stay resident: 671B weights at one byte each is 671 GB, on eight or more GPUs, to run 37B of compute per token. MoE buys FLOPs with memory.

Decode batching

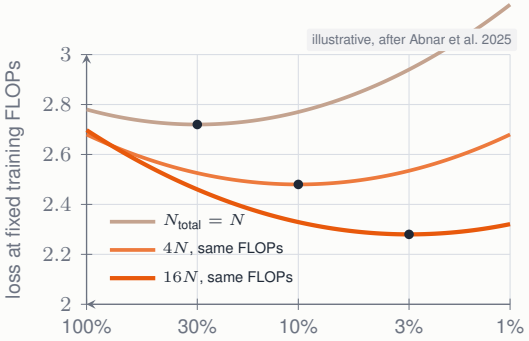
At batch 1 a token reads only its k experts, so bytes per token fall with the active fraction. A large batch touches every expert, and the layer looks dense again.

Load at inference

Expert popularity is uneven per domain; serving systems replicate hot experts, or place them so that each GPU's traffic is similar (DeepSeek's redundant experts).

MoE scaling laws: how sparse should a model be?

At a fixed training budget there is a best active fraction, and it falls as the total parameter count grows.



Moving left trades FLOPs per token for parameters; at a fixed budget the bigger total model should be the sparser one. Whether the optimum also moves with the budget is stated both ways in the paper.

What the papers found

Clark et al. 2022: a unified law in N and expert count; they recommend 64–128 experts, with diminishing returns beyond and a fitted saturation at a few hundred.

Krajewski et al. 2024: granularity enters the law; with it tuned, MoE beats dense at every scale, and the usual expert width is too coarse.

Abnar et al. 2025: with total parameters and FLOPs as separate axes, the compute-optimal sparsity rises with model size toward 1.

Ludziejewski et al. 2025: adding memory to the objective, MoE can be the memory-efficient choice too, not only the FLOP-efficient one.

In practice: 8 active of 128–384 fine-grained experts, one shared, active fraction 3–10%; the 2023 choice of 2-of-8 is now the coarse end.

A worked bill: DeepSeek-V3 against a dense model

Sparsity in the experts, compression in the attention, FP8 everywhere: the three levers in one model.

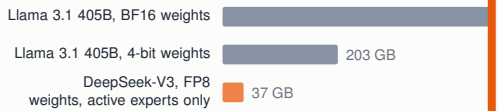
671B
total; 256 routed + 1 shared experts

37B
active per token: 8 routed + 1 shared

14.8T
tokens; 2.788M H800 GPU-hours, FP8

70 KB
KV per token, MLA, BF16 (derived)

bytes read per decoded token at batch 1 (derived)



plus the KV cache: 504 KB per token for Llama 3.1 405B (126 layers, 8 KV heads, $d_h = 128$, BF16) against 70 KB for V3 (61 layers, 512 latent + 64 rotary dims); at 100k tokens that is 52 GB against 7 GB.

Reading the bill

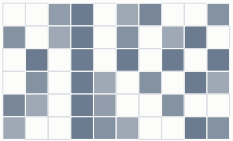
Training FLOPs $\approx 6 \cdot 37\text{B} \cdot 14.8\text{T} \approx 3.3 \times 10^{24}$, a 405B dense model's cost would be eleven times that per token.

At inference the 671 GB of FP8 weights must be resident, so the model needs a multi-GPU node; but each token reads 37 GB, so a batch of a few hundred tokens runs near the compute ridge.

The attention side is compressed, not sparse: MLA stores a 512-wide latent per token instead of 128 heads' keys and values. Sparse attention came later (V3.2, slide 16).

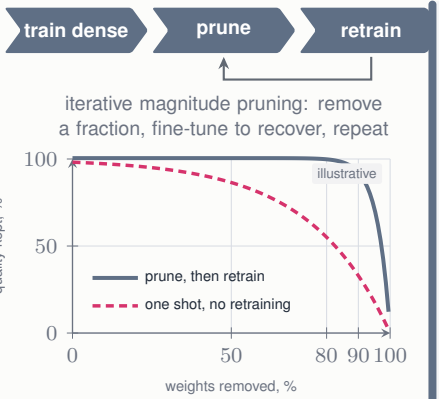
Weight pruning: the classics

Remove the weights that matter least, retrain, repeat. Most of a trained network is slack; the last 10% is not.



W with $|w|$ below a threshold removed (blank): 50% here

the cliff: quality holds to high sparsity with retraining and collapses early without. Where it sits depends on the criterion and the structure.



Which weights?

Magnitude $|w|$: Han et al. 2015 cut AlexNet 9× and VGG-16 13× with no accuracy loss, pruning and retraining.

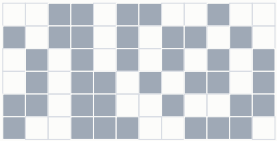
Second order: OBD (1989) scores $\delta\mathcal{L} \approx \frac{1}{2} H_{ii} w_i^2$; OBS (1993) uses the full inverse Hessian, $\delta\mathcal{L} = w_i^2 / (2[H^{-1}]_{ii})$, and compensates the others: the ancestor of SparseGPT.

Lottery tickets (Frankle & Carbin 2019): the surviving 10–20%, reset to their initial values, train alone to full accuracy; the mask matters, and dense training found it.

Structure decides the speedup

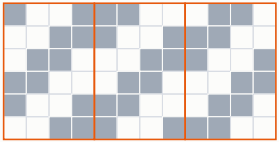
A zero only saves time if the hardware can skip it. Unstructured zeros rarely do on a GPU; 2:4 and whole blocks do.

unstructured



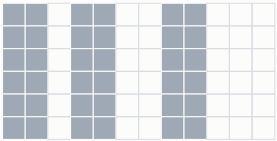
any weight can go; needs sparse kernels, and on GPUs the gain appears only past ∼90% zeros, so for LLMs it saves memory, not time. Best quality per removed weight.

semi-structured 2:4



two of every four along a row are zero, stored compressed with a 2-bit index; Ampere and later tensor cores run it at twice the dense rate: the "with sparsity" peak in every datasheet.

structured



whole neurons, heads or layers removed: a smaller dense model, fast anywhere, and the only kind that shrinks depth or the KV cache. Costs the most quality per weight; needs distillation.

pattern	what runs it	realistic LLM sparsity at small loss	what you get
unstructured	sparse kernels, CPUs, custom silicon	50–60% one-shot (SparseGPT, Wanda)	smaller checkpoint; little GPU speedup
2:4	NVIDIA sparse tensor cores	50% by construction; some loss at LLM scale	up to 2× matmul, ∼1.3–1.5× end to end
structured	everything	20–50% with distillation (Minitron, Sheared LLaMA)	a smaller model in every dimension

One-shot pruning at LLM scale: SparseGPT and Wanda

No retraining: prune one layer at a time so that its outputs on a few calibration samples barely change.

calibration set
128 samples $\times$ 2k tokens

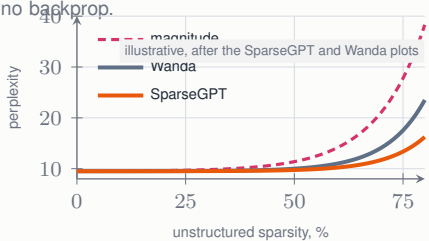
layer ℓ
inputs X, weights W

$\min_{\hat{W}} \|WX - \hat{W}X\|_2^2$
with the chosen zeros in $\hat{W}$

layer by layer, each seeing the pruned outputs of the previous one. The Hessian of this objective is $H = XX^\top$: local, cheap, no labels, no backprop.

SparseGPT (Frantar & Alistarh 2023): OBS applied column by column, sharing one inverse Hessian, with the error of each pruned column pushed onto the columns not yet processed. OPT-175B and BLOOM-176B to 50–60% unstructured, or 2:4, in under 4.5 hours on one GPU, with a small perplexity increase.

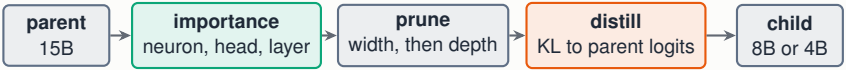
Wanda (Sun et al. 2023): score $S_{ij} = |W_{ij}| \cdot \|X_j\|_2$, keep the top entries per output row, update nothing. Nearly as good at 50%, in minutes; the activation norm is what magnitude pruning was missing, because of the outlier channels (slide 20).



flat to 50%, then the knee; bigger models have a later knee. Half the weights gone and nothing faster, unless 2:4.

Pruning with distillation, and a law for sparsity

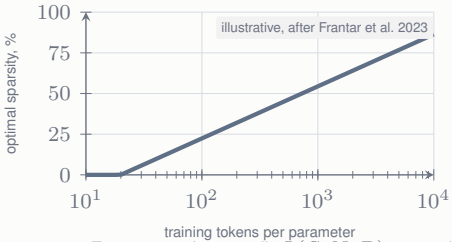
The cheap way to a small model is to cut a big one down and teach it from its parent, not to train from scratch.



Minitron (NVIDIA 2024): Nemotron-4 15B to 8B and 4B with up to 40× fewer training tokens than from scratch; width pruning hurt less than depth pruning, and distillation beat plain fine-tuning. Repeated for Llama-3.1-Minitron 4B.

Sheared LLaMA (Xia et al. 2023): LLaMA-2 7B to 2.7B and 1.3B with 50B tokens: learned masks, then continued pretraining on a reweighted mix, about 3% of the compute of training from scratch.

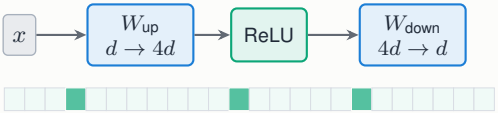
Why it works. The parent's search is done; the child keeps what the mask kept, and the parent's logits say exactly what that is.



Frantar et al. 2023 fit $L(S, N, D)$: sparsity discounts N, and the best sparsity rises with tokens per parameter; over-trained models are the ones worth pruning.

Activation sparsity: most neurons are silent for most tokens

In a ReLU MLP over 90% of the hidden units are zero per input; skip their rows and the layer is almost free.



$h = \mathrm{ReLU}(W_{\text{up}}x)$: three of 24 units fire; only their rows of W_{down} (and columns of W_{up}) are read.

Measured (Li et al. 2022): in T5 the share of nonzero MLP units per token is a few percent, and it falls as models grow. GELU and SiLU gates (Llama) give small values, not zeros, so the sparsity must be induced.

Induced. ReLUfication (Mirzadeh et al. 2023): swap the activation, fine-tune. Thresholding at inference (TEAL, CATS 2024): zero every unit below a magnitude, 40–50% across a Llama, no training. Deja Vu (Liu et al. 2023): predict the live units and heads from the input with a small side network, 85% on OPT-175B.

Where the time goes

Skipping a unit saves its row of W_{down} and column of W_{up}: at decode, where the layer is bound by reading weights, that is the whole cost, so 50% sparsity is up to 2× on the MLP; TEAL reports 1.5–1.8× end to end. At prefill the matmuls are dense and batched, and nothing is saved.

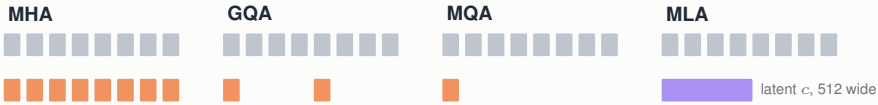
Predict or test? Testing the unit needs the up-projection first; Deja Vu predicts the set from the previous layer, overlapping the lookup with compute.

MoE is this, made structured: experts are groups of units that switch on together, chosen by a router rather than by a threshold, so the sparsity is known before the matmul and batches cleanly.

Attention: where the cost is, and the KV cache

Prefill is quadratic in length; decode reads the whole cache per token. Both are attacked by sharing and by sparsity.

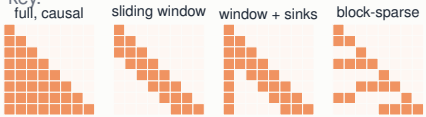
prefill FLOPs per layer $\approx 4\,L^2 d$ (scores and values) decode bytes per token per layer $= 2\,h_{kv}\,d_h \cdot \text{bits}/8 \cdot L$



top: query heads; bottom: cached per token. GQA (Llama 3): 8 KV heads for 64–128 query heads; MQA: one; MLA (DeepSeek-V2/V3): a 512-wide latent plus a 64-wide rotary key.

model	layers	KV heads	d_h	KB/token, BF16	at 128k
Llama 3 8B	32	8	128	128	17.2 GB
Llama 3 70B	80	8	128	320	42.9 GB
Llama 3.1 405B	126	8	128	504	67.6 GB
DeepSeek-V3 (MLA)	61	latent 512+64	–	69	9.2 GB

derived from the published configs; KB $= 1024$ bytes. The cache, not the weights, runs out first at long context.



fixed patterns: Sparse Transformer, Longformer, BigBird (2019–20); Mistral 7B's 4,096-token window; StreamingLLM's 4 sink tokens plus a window. Cheap, static, blind to content.

Learned sparse attention: pick the tokens that matter

Score cheaply, attend fully to the top k: the 2025 designs make long context linear in what is read.

DeepSeek Sparse Attention (V3.2, 2025)



per query: an $O(L)$ indexer with tiny heads, then $O(k)$ attention; memory unchanged, reads cut by L/k.

Fixed patterns are blind to content. Eviction is cheap but irreversible: a dropped token is gone for every later query. Selection keeps everything and chooses per query, at the price of an indexer and a cache that still grows with L. Below ∼8k tokens dense attention is already cheap next to the MLPs; the gain is at decode and in long prefill.

NSA (DeepSeek, Feb 2025)

Three branches summed with learned gates: compressed block summaries, the top-n selected blocks, and a sliding window. Kernels written for the pattern; trained from scratch; large speedups at 64k on decode, forward and backward.

MoBA (Moonshot, Feb 2025)

The KV cache in blocks; each query is routed to its top blocks by a mean-key score, as a router routes tokens to experts. Switchable between full and sparse attention at inference.

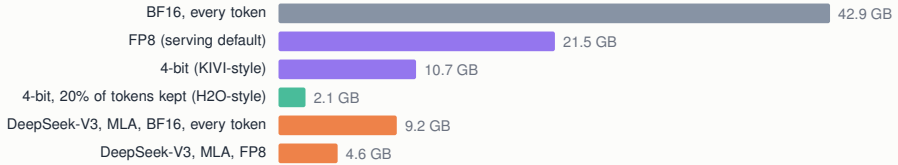
Eviction, no training (2023–24)

H2O keeps the heavy-hitter tokens (20% budget); StreamingLLM keeps the sinks and a window; SnapKV compresses the prompt's cache; Quest picks pages per query. Memory and reads both shrink.

The KV cache is where sparsity and precision meet

Fewer heads, fewer bits, fewer tokens: the three multiply, and the cache at 128k goes from 43 GB to about 2.

KV cache for one 128k-token sequence, Llama 3 70B (GQA, 8 KV heads), derived



eviction (H2O, SnapKV) shrinks memory and reads; selection (DSA, NSA, Quest) shrinks reads only, the cache still holds every token: at $k = 2048$, V3.2 reads about 140 MB of latent per step instead of 9.2 GB (derived).

Precision in the cache

FP8 keys and values are routine in vLLM and TensorRT-LLM. Keys carry outlier channels, so KIVI quantizes keys per channel and values per token, down to 2–4 bits.

Sparsity in the cache

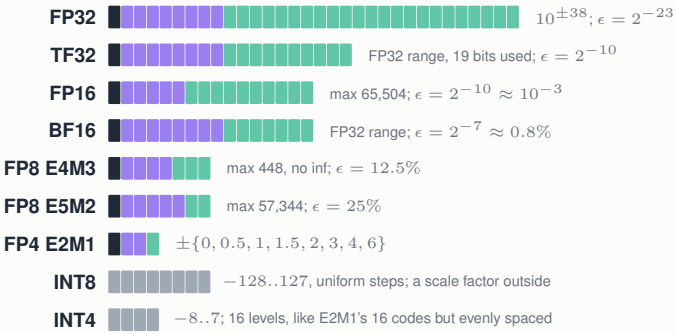
Heads: GQA 8×, MLA ∼7× over MHA. Tokens: eviction by attention mass (H2O, SnapKV) or per-query selection (Quest, DSA). 4-bit at a fifth of the tokens is 20×.

What to expect

GQA plus FP8 covers most serving. Past 128k the cache dominates even at FP8; 2025–26 models add selection or MLA compression. Judge quality on long-context evals, not perplexity.

Anatomy of a number

Exponent bits buy range, mantissa bits buy precision; every format is a different split of a fixed budget.



■ sign ■ exponent ■ mantissa. Value $= (-1)^s\, 2^{e-\text{bias}}\,(1 + m/2^M)$; $\epsilon = 2^{-M}$ is the relative step between neighbours.

Reading the table

Eight exponent bits (FP32, BF16) reach $10^{\pm 38}$; five (FP16, E5M2) reach $6 \cdot 10^4$; four (E4M3) reach 448; two (E2M1) reach 6. Range is what gradients need; precision is what weights and activations need.

TF32 and BF16 are FP32 with the mantissa cut, so they need no loss scaling. E4M3 spends its codes on precision and gives up infinities; E5M2 is FP16 with the mantissa cut.

Below 8 bits the per-number precision is so coarse that the format only works with a shared scale per small block (next slide).

Scale factors: where the lost bits go

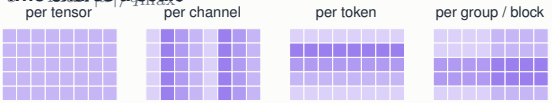
A low-precision number is a code times a scale. The question is how many numbers share one scale.

the 8 positive codes of E2M1 against 8 of INT4, both scaled to a max of 6



floats put their codes near zero, where most weights live; integers waste codes at the tails but are exact for the large values. Both need a scale s: $x \approx s \cdot q$, $q = \mathrm{round}(x/s)$, with $s = \max|x| / q_{\max}$.

who shares a scale



one color = one scale. Weights: per output channel or per group of 128 along the input (GPTQ, AWQ). Activations: per token, because the outlier channels (next slide) would set everyone's scale. Blocks of 32 or 16 are small enough for 4-bit codes.

Block formats, and the real bits per number

MX (OCP spec, 2023): blocks of 32 share one 8-bit power-of-two scale (E8M0). MXFP8: E4M3 codes, 8.25 bits each; MXFP4: E2M1 codes, 4.25 bits. Native on Blackwell.

NVFP4 (NVIDIA, 2025): blocks of 16 with an E4M3 scale, plus one FP32 scale per tensor: 4.5 bits per number, and a finer scale that loses less to rounding.

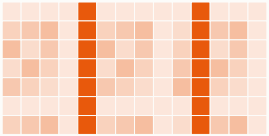
INT4, group 128 with an FP16 scale and zero point: 4.16 bits (GPTQ, AWQ). **NF4** (QLoRA): 16 levels at the quantiles of a normal.

Rounding noise has variance $s^2/12$; each bit removed doubles s and costs 6 dB of signal-to-noise, unless the block is small enough to keep s small.

Outliers: the few channels that set everyone's scale

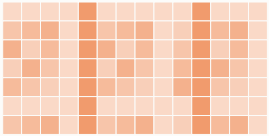
A few hidden dimensions carry values far larger than the rest; every low-precision method works around them.

activations X (tokens $\times$ channels)



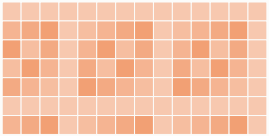
two channels dominate and set every per-token scale

after SmoothQuant: $X\,\mathrm{diag}(s)^{-1}$



divide channel j by s_j, scale W's column j: exact

after a Hadamard rotation XH



orthogonal H, with $H^\top$ folded into W: outliers spread out

LLM.int8() (Dettmers et al. 2022): the outliers appear at ∼6.7B parameters in fixed hidden dimensions; keep those columns in FP16, the rest in INT8, and nothing is lost up to 175B.

SmoothQuant (Xiao et al. 2022): $s_j = \max|X_j|^\alpha / \max|W_j|^{1-\alpha}$, $\alpha = 0.5$; one offline rescaling, then plain W8A8 matmuls.

Rotations (QuaRot, SpinQuant 2024): an orthogonal H, random or learned, also inside attention, flattens enough for 4-bit activations and KV cache; on gradients the same trick makes FP4 training work (slide 24).

Why they exist

The outlier channels act as attention sinks and norm anchors: a few dimensions the network uses as constants, trained to huge values so that layer norm and softmax behave. They are a feature, not noise, which is why pruning them or rounding them is what breaks a quantized model, and why Wanda weighs weights by the activation norm.

What the tensor cores run, by generation

Each generation halves the smallest native format and doubles its throughput; bandwidth grows slower than FLOPs.

GPU (year)	new native formats	BF16	FP8	FP4	HBM	bandwidth	ridge (BF16)
		dense tensor TFLOP/s			GB	TB/s	FLOP/byte
A100 (2020)	TF32, BF16, INT8; 2:4 sparsity	312	–	–	80	2.0	156
H100 SXM (2022)	FP8 E4M3/E5M2 (Transformer Engine)	989	1,979	–	80	3.35	295
B200 (2024)	FP4 (NVFP4, MXFP4), FP6; 2nd-gen TE	2,250	4,500	9,000	180	8.0	281
MI300X (2023)	FP8	1,307	2,615	–	192	5.3	247
MI355X (2025)	FP4, FP6	2,520	5,030	10,070	288	8.0	315
Rubin (2026)†	NVFP4 training at 35 PFLOP/s, HBM4	–	–	35,000	288	22	–

vendor datasheet peaks, dense (headline numbers are usually "with sparsity", twice these); B200 per GPU is derived from the 8-GPU HGX totals; Rubin from NVIDIA's 2026 announcement. Ridge = BF16 peak over bandwidth, derived.

† t

Format support is the lever

A format pays only when the tensor core runs it natively: INT8 on Ampere, FP8 on Hopper, FP4 on Blackwell. Each moved the serving default.

FLOPs outrun bytes

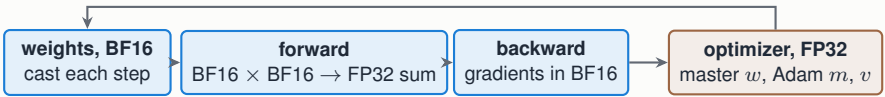
A100 to B200: dense BF16 up 7×, bandwidth 4×. Decode gets more memory-bound every generation; the case for fewer bits grows with it.

Accumulation is the fine print

Products are summed in a wider accumulator; how wide matters. DeepSeek-V3 measured ∼14 bits on H800's FP8 path and moved partials to FP32 every 128 terms.

Mixed-precision training: master weights, loss scaling, BF16

Compute in 16 bits, accumulate and update in 32: the recipe since 2017, and why BF16 made it simpler.



bytes per parameter: 2 (BF16 weights) + 2 (gradients) + 4 + 4 + 4 (FP32 master, m, v) = 16, plus activations; the FP32 copy is where small updates survive.

Why a master copy. An update ηg is often 10^{-4} of the weight or less; in 16 bits $w + \eta g$ rounds back to w and training stalls. Adding in FP32 and re-casting keeps every step.

Loss scaling (Micikevicius et al. 2018): FP16's smallest normal is $6 \cdot 10^{-5}$ and most gradients are below it: multiply the loss by S, unscale before the update, lower S on overflow.

BF16 keeps FP32's exponent, so nothing underflows and loss scaling goes; the price is 7 mantissa bits, which the FP32 master absorbs. Default on TPUs and from Ampere on.

What stays in 32 bits

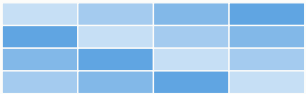
Matmuls run in the low format with FP32 accumulation. Softmax, layer norm, residual adds, the loss, and the optimizer state stay in FP32, or BF16 with care. Gradient all-reduce often goes in BF16, and in FP8 in some systems.

Memory is the pressure: DeepSeek-V3 keeps Adam's m and v in BF16 and only the master weights and gradient accumulation in FP32, which brings 16 bytes per parameter down to 12.

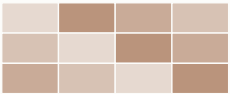
FP8 training: the scale goes per tensor, then per tile

Eight bits hold a GEMM if each small block has its own scale and the sums are carried in 32.

activations X: scale per 1×128 tile **weights W: scale per 128×128 block**



rows: tokens; 128 columns per tile along K



the scales of the tile and the block multiply the partial sum on the way to FP32 (DeepSeek-V3, Sec. 3.3)

Transformer Engine (NVIDIA, 2022): one scale per tensor from the recent history of maxima (“delayed scaling”); E4M3 for weights and activations, E5M2 for gradients; FP32 master and optimizer.

DeepSeek-V3 (2024), the first open 600B-class model trained this way: E4M3 everywhere; activation scales per 1×128 tile, weight scales per 128×128 block, so an outlier spoils only its tile; partial sums promoted to FP32 every 128 products, after measuring about 14 bits of accumulation on H800; the rest stays in BF16 or FP32. Loss within 0.25% of a BF16 run.

MXFP8 (NVIDIA, 2025): E4M3 with an E8M0 scale per 32, the scale rounded up toward $+\infty$ rather than down as in the OCP spec; an 8B model on 15T tokens matched BF16.

FP4 training arrives

Four bits need everything the FP8 recipe has, plus unbiased rounding and a rotation before quantizing.



each step halves the matmul input bits and doubles throughput; each needed a new trick to keep the gradients.

2D block scaling for weights

a 16×16 block shares its scale, so W and $W^\top$ quantize identically in the forward and backward passes.

Random Hadamard transforms

rotate gradients (and activations) before quantizing so no single outlier sets a block's scale; undone exactly after the matmul.

Stochastic rounding

round up or down with probability given by the remainder: unbiased gradients, where round-to-nearest pushes small updates to zero.

Keep the sensitive layers

embeddings, norms, the output head and the first and last blocks stay in BF16 or FP8; NVFP4 pretraining kept 16% of the linear layers higher.

What was shown

NVFP4 (NVIDIA 2025): 12B on 10T tokens, loss gap 1–1.5% vs FP8, tasks matched*. MXFP4 (Tseng 2025): near-lossless with SR + RHT.

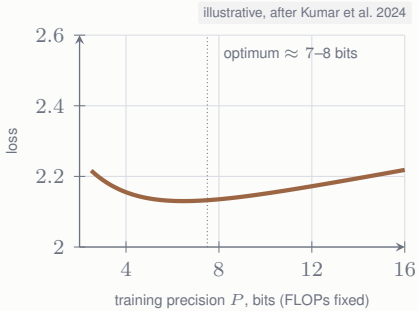
Where it stands

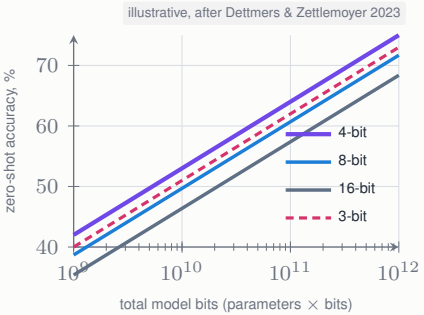
Nemotron 3 Ultra (June 2026): 550B/55B pretrained in NVFP4 on 20T tokens, loss gap under 0.4%†. DeepSeek-V4 ships expert weights in FP4†.

*numbers as reported by the authors; †from vendor and lab write-ups, see the sources slide.

Scaling laws for precision

Bits are a resource like parameters and tokens: there is a compute-optimal precision, and it is not 16.





Training (Kumar et al. 2024): $N_{\text{eff}} = N(1 - e^{-P/\gamma})$ in the Chinchilla form; at fixed FLOPs, lower P buys more N until ∼7–8 bits, then the discount wins.

Inference (Dettmers & Zettlemoyer 2023): across 19M–176B parameters, 4-bit weights give the best accuracy per total bit; 3-bit loses; holds across families.

Also

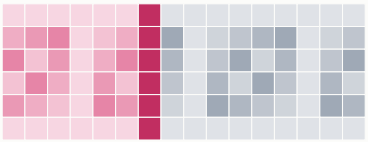
PTQ hurts more the more tokens per parameter a model saw: δ_{PTQ} grows with D/N in the fit; an over-trained 8B is the hardest to quantize.

Sun et al. 2025 extend the law to float formats.

Post-training quantization I: weights only

Round the weights to 4 bits after training, one layer at a time; decode gets faster because it reads fewer bytes.

GPTQ's column sweep over W



done (4-bit) · current column q · not yet: receives the correction $-\dfrac{w_q - \mathrm{quant}(w_q)}{[H^{-1}]_{qq}}\,[H^{-1}]_{q,>q}$

GPTQ (Frantar et al. 2022): OBS on the layer-wise objective of slide 12 with $H = 2XX^\top$, columns in batches of 128 and a Cholesky inverse; OPT-175B to 3–4 bits in about 4 GPU-hours.

AWQ (Lin et al. 2023): the $\sim$1% of channels with the largest activations carry the loss; scale them up before rounding (Ws, X/s), with s searched per channel on calibration data; no backprop; 4-bit, groups of 128.

Why it is faster, and when

At batch 1 decode reads every weight once per token, so 4-bit weights cut the bytes 4$\times$ against BF16 and the step time with them; the matmul itself dequantizes on the fly (Marlin, 2024: near-ideal speedups up to batch 16–32). Past the ridge (slide 2) the layer is compute-bound and weight-only quantization gains nothing; serving at large batch wants activations quantized too.

Formats in the wild. GPTQ and AWQ checkpoints on the Hub; llama.cpp's k-quants at 2–6 bits; NF4 for fine-tuning (QLoRA); all with per-group scales, 4.1–4.6 real bits per weight.

Below 4 bits codebooks take over: QuIP# and AQLM (2024) reach 2 bits with vector quantization and rotations, at a cost in quality that grows with how over-trained the model is.

Post-training quantization II: activations too

Weight bits help decode; activation bits help prefill and large batches. Hardware decides which pairs run natively.

weight bits × activation bits, and who runs it

	A16	A8	A4
W16	BF16 baseline	–	–
W8	INT8 weight-only; rare	INT8 (SmoothQuant); FP8 W8A8: the serving default	–
W4	GPTQ, AWQ, k-quants; the local default	QServe (W4A8KV4), Atom	QuaRot, SpinQuant; NVFP4 native on Blackwell
W2	QuIP#, AQLM codebooks	–	–

a cell is natively fast when its pair matches a tensor-core format: INT8 since Ampere, FP8 since Hopper, FP4 since Blackwell; otherwise it is a memory trick with dequantization in the kernel.

The two halves of serving

Prefill and large-batch decode are compute-bound: only activations in a native low format buy speed there. FP8 W8A8 with per-token dynamic scales is near-lossless and about twice BF16 on H100; INT8 needs SmoothQuant-style calibration; 4-bit activations need the rotations of slide 20.

Small-batch decode is bandwidth-bound: weight bits are what count, activations can stay at 16 bits (W4A16).

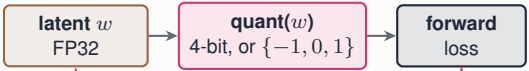
Blackwell makes W4A4 native (NVFP4, 16-element scales); NVIDIA reports accuracy close to FP8 on common benchmarks for PTQ-quantized models†. The KV cache follows the same split (slide 17): per-token scales, FP8 routine, 4-bit with care.

DeepSeek-V3 ships FP8 weights from training: no PTQ step, and the serving precision equals the training precision.

QAT, ternary weights, and quantized experts

When post-training rounding is not enough, train with the rounding in the loop.

the straight-through estimator



backward: treat the rounding as the identity and pass $\partial \mathcal{L} / \partial \operatorname{quant}(w)$ straight to w; the latent weights move in FP32, the forward sees only rounded values

QAT (LLM-QAT 2023; Llama 3.2 1B/3B, Meta 2024): a short fine-tune with quantization in the loop, often through LoRA adapters (Meta's "QLoRA"); Meta reports 2–4× faster decode, 56% smaller models and 41% less memory than BF16, with SpinQuant as the no-training alternative.

BitNet b1.58 (Ma et al. 2024): every weight in $\{-1, 0, 1\}$ from the start, activations in 8 bits; the matmul becomes additions. Reported to match an FP16 LLaMA from 3B at the same tokens; 2B4T (2025) trained on 4T tokens.

Quantized experts

MoE weights are nearly all expert weights that each token mostly does not touch, and they quantize well.

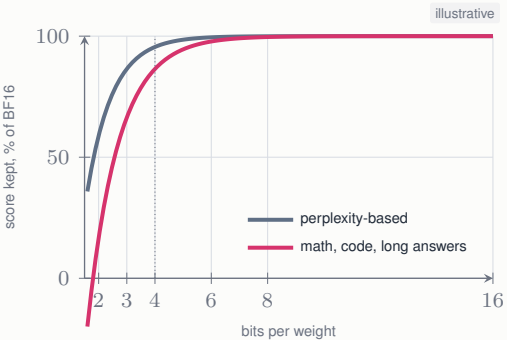
gpt-oss (OpenAI 2025): the expert weights post-trained into MXFP4 (4.25 bits), the attention and embeddings left in BF16; the 117B model fits one 80 GB GPU and the 21B one runs in 16 GB.

DeepSeek-V3: FP8 from training, 671 GB; community 4-bit conversions bring it near 350 GB, with the usual loss on the hardest tasks.

The router stays in high precision: a few thousand numbers whose rounding would change which experts fire, and with them the whole computation.

Measuring the damage

Perplexity is the wrong instrument for a 4-bit model; the losses show up in long generations and hard tasks.



the knee at 4 bits is real on both curves; what moves is where the cliff starts, and task-level scores fall earlier than perplexity suggests.

A checklist

Evaluate what you serve: long generations, sampled many times, on math and code, where rounding errors compound over thousands of tokens; long-context tasks if the KV cache is quantized.

Count the bits honestly: INT4 with group-128 FP16 scales is 4.16, MXFP4 4.25, NVFP4 4.5; compare models at equal total bits, where a larger model at fewer bits usually wins (Dettmers & Zettlemoyer).

Expect the tax to grow with tokens per parameter (Kumar et al.): a 15T-token 8B loses more at 4 bits than a 1T-token 8B.

Check the calibration set: GPTQ, AWQ and SmoothQuant fit to a few hundred samples; a domain shift there is a quality shift later.

Report speed at your batch: peak sparse TFLOP/s and batch-1 tokens per second describe different machines.

Serving three models: the bill, with every lever applied

A dense 405B, a sparse 671B and a sparse-and-quantized 117B, as they ship.

model	total / active	weights as shipped	weight bytes	80 GB GPUs	bytes per token, batch 1	KV per token
Llama 3.1 405B	405B / 405B	BF16 (FP8 variant)	810 GB (405)	11 (6)	810 GB (405)	504 KB
DeepSeek-V3	671B / 37B	FP8 from training	671 GB	9+	37 GB	69 KB
gpt-oss-120b	117B / 5.1B	MXFP4 experts, BF16 rest	∼65 GB	1	∼3 GB	96 KB*

derived from the published configurations; GPU counts ignore the KV cache and activations, so real deployments use more. *gpt-oss-120b: 36 layers, 8 KV heads, $d_h = 64$, alternating full and 128-token banded attention, so most layers' caches stop growing.

What sparsity bought

V3 computes 37B per token and reads 37 GB; a dense 405B reads 810 GB. gpt-oss reads ∼3 GB: a 117B model at the decode cost of a 5B one.

What precision bought

FP8 halved V3's footprint for free, because it trained that way. MXFP4 put 117B on one GPU; at BF16 it would be 234 GB on three.

What they still cost

Every expert must be resident: V3 needs a node however few tokens it serves. Its 69 KB per token of MLA cache keeps 128k contexts affordable.

the pattern of 2025: train sparse (MoE, MLA) in FP8, ship the experts at 4 bits, and attack the cache with sharing, FP8 and selection; the three levers compose multiplicatively on the bytes per token.

A practical guide

What to run, what to train in, and what to build, as of 2026.

Inference: what to run

Default: FP8 W8A8 on Hopper or Blackwell, per-token scales; near-lossless, about 2× BF16.

Memory-bound or small batch: 4-bit weight-only (AWQ, GPTQ, k-quants) with a fused kernel; keep activations at 16.

Blackwell: NVFP4 W4A4 once validated on your tasks.

Cache: FP8 KV always; GQA or MLA; selection or eviction past 128k.

Always: evaluate the quantized model on long generations, not perplexity.

Training: what to train in

Default: BF16 compute, FP32 master and optimizer.

FP8 when the stack supports per-block scales (Transformer Engine, the DeepSeek-V3 recipe): E4M3, tiles of 128, FP32 accumulation; expect 1.5–2× on the matmuls, not end to end.

FP4 is a research setting in 2026: SR, Hadamard, 2D block scales, sensitive layers kept higher; run a BF16 control on the same tokens.

Watch loss spikes and the gradient-norm history; they are the first symptom.

Architecture: what to build

MoE when you serve at scale and have the memory: fine-grained experts, $k = 8$ of 128 or more, a shared expert, aux-loss-free balance, expert parallelism with overlapped all-to-all.

Dense when memory is the limit or the model is small.

Small models by pruning and distilling a larger one, not from scratch.

Long context with GQA or MLA plus a selection mechanism; fixed windows only for the cheap tier.

Plan the bits early: a model trained in FP8 ships in FP8.

Summary: one cost model, two levers

Every method on these slides changes either how many numbers a token touches or how many bits each number has.

MoE	pruning	act./attn.	formats	training	quantization
buys: capacity without FLOPs; 3–10% active	**buys:** smaller checkpoints; small models from big	**buys:** decode and long context linear in what is read	**buys:** the right split of range and precision	**buys:** 2× per generation (FP8, FP4)	**buys:** 4× fewer bytes at decode
costs: memory; routing; all-to-all	**costs:** no speedup unless structured	**costs:** predictors, indexers, evals	**costs:** outliers; accumulation width	**costs:** scales, rounding, stability	**costs:** task-level damage; honest bits

Six things to remember

1. Decode is memory-bound by two orders of magnitude; bytes per token, not FLOPs, is the number to cut, and both levers cut it. **2.** MoE trades memory for compute: quality follows total parameters at a discount, cost follows active ones; the optimal active fraction falls as models grow, and the open models went from 1/4 to 1/30 in two years. **3.** Zeros are free only when the hardware can skip them: 2:4 and structured pruning run fast, unstructured pruning saves memory; the small models worth having are pruned and distilled from big ones. **4.** Outlier channels are the whole difficulty of low precision; per-block scales, smoothing and rotations are three ways to defuse them, and the same rotation lets FP4 training work. **5.** Precision has a scaling law: about 8 bits is compute-optimal for training, 4 bits is the knee for inference, and over-trained models pay the largest quantization tax. **6.** The frontier open recipe composes all of it: sparse experts and compressed attention trained in FP8, shipped at 4–8 bits, with a cache that is shared, quantized and selectively read.

Sources

Primary papers, checked against arXiv; † marks claims taken from vendor pages, model cards, blogs or commentary.

Mixture of experts. Shazeer et al. 2017, sparsely-gated MoE, 1701.06538. Lepikhin et al. 2020, GShard, 2006.16668. Fedus et al. 2021, Switch, 2101.03961. Du et al. 2021, GLaM, 2112.06905. Zoph et al. 2022, ST-MoE, 2202.08906. Zhou et al. 2022, expert choice, 2202.09368. Clark et al. 2022, routed scaling laws, 2202.01169. Gale et al. 2022, MegaBlocks, 2211.15841. Jiang et al. 2024, Mixtral, 2401.04088. Dai et al. 2024, DeepSeekMoE, 2401.06066. Krajewski et al. 2024, fine-grained MoE laws, 2402.07871. DeepSeek-V2, 2405.04434. Wang et al. 2024, aux-loss-free balancing, 2408.15664. DeepSeek-V3, 2412.19437. Abnar et al. 2025, optimal sparsity, 2501.12370. Ludziejewski et al. 2025, joint MoE laws, 2502.05172. Qwen3, 2505.09388. Kimi K2, 2507.20534. gpt-oss model card, 2508.10925; Llama 4, Qwen3-Next: Meta and Qwen blogs and HF cards†.

Pruning and sparsity. LeCun et al. 1989, OBD. Hassibi & Stork 1993, OBS. Han et al. 2015, 1506.02626. Frankle & Carbin 2019, 1803.03635. Mishra et al. 2021, 2:4, 2104.08378. Frantar & Alistarh 2023, SparseGPT, 2301.00774. Sun et al. 2023, Wanda, 2306.11695. Frantar et al. 2023, sparse scaling laws, 2309.08520. Xia et al. 2023, Sheared LLaMA, 2310.06694. Muralidharan et al. 2024, Minitron, 2407.14679; Sreenivas et al. 2024, 2408.11796. Li et al. 2022, lazy neurons, 2210.06313. Mirzadeh et al. 2023, ReLU strikes back, 2310.04564. Liu et al. 2023, Deja Vu, 2310.17157. Liu et al. 2024, TEAL, 2408.14690. Lee et al. 2024, CATS, 2404.08763.

Attention. Child et al. 2019, 1904.10509. Beltagy et al. 2020, Longformer, 2004.05150. Zaheer et al. 2020, BigBird, 2007.14062. Shazeer 2019, MQA, 1911.02150. Ainslie et al. 2023, GQA, 2305.13245. Jiang et al. 2023, Mistral 7B, 2310.06825. Zhang et al. 2023, H2O, 2306.14048. Xiao et al. 2023, StreamingLLM, 2309.17453. Li et al. 2024, SnapKV, 2404.14469. Tang et al. 2024, Quest, 2406.10774. Yuan et al. 2025, NSA, 2502.11089. Lu et al. 2025, MoBA, 2502.13189. DeepSeek-V3.2, 2512.02556 (V3.2-Exp, Sept 2025, GitHub†). DeepSeek-V4, 2606.19348; Kimi K3 model card†. Hsieh et al. 2024, RULER, 2404.06654.

Formats and hardware. Micikevicius et al. 2018, mixed precision, 1710.03740. Micikevicius et al. 2022, FP8 formats, 2209.05433. OCP Microscaling Formats spec v1.0, 2023†. NVIDIA, NVFP4 (developer blog, 2025)†. Dettmers et al. 2023, QLoRA/NF4, 2305.14314. GPU figures: NVIDIA A100, H100, HGX B200 datasheets; AMD MI300X, MI355X pages†.

Low-precision training. NVIDIA Transformer Engine docs†. DeepSeek-V3, Sec. 3.3, 2412.19437. Tseng et al. 2025, MXFP4 training, 2502.20586. NVIDIA 2025, MXFP8 recipes, 2506.08027. NVIDIA 2025, NVFP4 pretraining, 2509.25149. Castro et al. 2025, Quartet, 2505.14669. NVIDIA, Nemotron 3 Ultra technical report, June 2026†; Rubin announcement, 2026†.

Scaling laws. Kumar et al. 2024, precision, 2411.04330. Dettmers & Zettlemoyer 2023, 4-bit, 2212.09720. Sun et al. 2025, FP quantization training, 2501.02423.

Quantization. Dettmers et al. 2022, LLM.int8(), 2208.07339. Xiao et al. 2022, SmoothQuant, 2211.10438. Frantar et al. 2022, GPTQ, 2210.17323. Lin et al. 2023, AWQ, 2306.00978. Ashkboos et al. 2024, QuaRot, 2404.00456. Liu et al. 2024, SpinQuant, 2405.16406. Liu et al. 2024, KIVI, 2402.02750. Lin et al. 2024, QServe, 2405.04532. Frantar et al. 2024, Marlin, 2408.11743. Tseng et al. 2024, QuIP#, 2402.04396. Egiazarian et al. 2024, AQLM, 2401.06118. Liu et al. 2023, LLM-QAT, 2305.17888. Ma et al. 2024, BitNet b1.58, 2402.17764; 2B4T, 2504.12285. Meta, Llama 3.2 quantized models (blog, Oct 2024)†. OpenAI, gpt-oss (blog and card, Aug 2025)†.

Curves labelled *illustrative* are sketches of a mechanism, not data; numbers labelled *derived* were computed from published configurations (KB $= 1024$ bytes, GB $= 10^9$ bytes). Hardware peaks are dense unless stated and come from vendor datasheets.